

Streamed DPO on a free Colab T4: DON'T SHIP

Run. Colab Tesla T4 (sm_75, fp16 only), soup-cli 0.75.1, torch 2.14, transformers 5.17, peft 0.21.1, trl 0.29.1. Qwen2.5-1.5B-Instruct @ 989aa79, DPO β 0.1, LoRA r16 on q_proj, v_proj (Soup's auto), lr 5e-5, batch 2 $\times$ accum 4, 63 steps, max_length 1024, stream_layers: true (2.62 GB pinned store, 2 $\times$ 94 MB layer buffers). Data: 500 pairs from 0x7o/oasst-ru-dpo-v1 (real Russian OpenAssistant text) + 100 held-out pairs never trained on (data/README.md). Every number below is in logs/t4/ (UTC-timestamped) or nvidia-smi/; details in [APPENDIX.md](#).

Two setup failures, kept in the logs. Colab now runs Python 3.13 and Soup $\geq$ 0.73.0 declares <3.13, so the task's own pip install "soup-cli[train]" quietly resolves to **0.72.4** (exit 0, no warning); I used 0.75.1 in a 3.12 venv. The first training run then stopped at setup: Colab's HF cache links each blob into a shared store and Soup's snapshot check (spectrum_scan.py:690) refuses that second hop. Fix: the same revision in a plain directory, only base: changed.

1. Memory budget, predicted before training vs measured

scripts/memory_budget.py (logged before step 1) predicted **11.11 GiB** allocated. Weights are small here: tied embedding 0.44, two layer buffers 0.17, LoRA + grads + Adam + TRL's frozen ref snapshot 0.04, activation checkpoints and one recompute 0.60 GiB. **Logits are 83%:** TRL keeps the policy logits plus a shifted copy for backward, then the reference forward makes two more while those are alive. Each copy is 4 rows $\times$ 1024 $\times$ 151,936 $\times$ **4 bytes** (a probe showed the logits reaching the loss are fp32): 2.32 GiB $\times$ 4 = 9.28 GiB, plus a 0.58 GiB log-softmax temp.

Soup pre-flight	My budget	max_memory_allocated	max_memory_reserved	nvidia-smi peak
9.99 GB = 9.30 GiB	11.11 GiB	11.41 GiB	12.41 GiB	12.58 GiB

The gap. Soup's pre-flight was 23% low on this DPO step (17% low locally, where it admitted a run that then ran out of memory in the reference pass). It is documented as never under-predicting; it prices paired losses like SFT and does not count the reference logits. My budget is 2.6% low (0.30 GiB unattributed). Reserved – allocated (1.0 GiB) is fragmentation: every batch pads to a different length. nvidia-smi adds the CUDA context (0.17 GiB). Streaming saved only 2.3 GiB here; the model would fit resident. The lever at this shape is the logits (precomputed reference log-probs, which TRL supports and Soup does not expose, or a chunked log-softmax), not the weights.

2. Proving the model trained

Why the loss is not proof. It ran 0.6931 $\rightarrow$ 0.705 (last step), mean 0.6685: at batch 8 it is mostly noise. Even a clean fall is measured inside the training process on training pairs: it cannot show the saved file carries the change (Soup has shipped two bugs that saved streamed adapters empty), that the adapter is active when loaded, that the model generalises, or that it did not learn a shortcut. One fp16 step also overflowed and was skipped (grad_norm: nan).

The check (scripts/verify_training.py): fresh process, saved file, fp32, 100 held-out pairs. A: the file has the 112 LoRA tensors and every lora_B (zero at init) moved. B: every tensor loads and the adapter moves the logits. Verdict: the DPO implicit-reward margin favours the chosen answer on held-out pairs **more often than a pure length heuristic would**, i.e. length-balanced accuracy (mean of the chosen-longer and chosen-shorter slices) with 95% CI above 0.5, in Qwen's own chat template (the serving format). Controls it must reject: **null** (lora_B zeroed, a run that saved nothing) and **random** (noise of the same norm: weights change, nothing learned).

Adapter	ΔW rel.	logit shift	train acc	held-out balanced acc [95% CI]	longer / shorter	verdict
main run	1.3e-3	2.22	0.81	0.49 [0.40, 0.58]	0.72 / 0.26	FAIL
null control	0	0.00	0.00	0.00 (all ties)	0 / 0	rejected
random control	1.3e-3	0.15	0.49	0.55 [0.45, 0.65]	0.45 / 0.66	rejected
catalog-lr run (5e-6)	1.3e-4	0.58	0.79	0.51 [0.42, 0.60]	0.75 / 0.26	FAIL
fix run (see §4)	1.1e-3	0.58	0.83	0.48 [0.38, 0.57]	0.69 / 0.26	FAIL

The run is real: a live adapter, as large a weight change as the random control but coherent (logit shift 2.22 vs 0.15), and TRL's reference snapshot was verified all-zero, i.e. the base. What it learned does not generalise: 81% on training pairs, and on unseen pairs it prefers the longer answer (72% right when chosen is longer, 26% when shorter).

My first rule was wrong, and a control caught it. It passed when plain held-out accuracy cleared 0.5. But 65 of the 100 held-out pairs have the longer answer chosen, so a model that only learned "longer wins" scores ~65% and passes. The catalog-lr run did pass it (lower bound 0.502) while scoring 0.26 on chosen-shorter pairs. The balanced rule is 0.5 for any pure length heuristic; the superseded rule is still printed in every output.

What the check misses. Whether the labels are right (the dataset card is empty); what the model generates (it scores fixed texts); other shortcuts (markdown, politeness); a wrong-but-moving backward pass (Soup's #331 class); real support tickets; effects under ± 10 points (n = 100); serving paths other than PEFT + transformers.

3. Silent failures (a run that finishes but is wrong)

What goes wrong	How checked	Soup catches it?
Model learns "longer wins", not the labels (69.6% of chosen are longer)	verify, length slices	data lint says MINOR ; the run proceeds

What goes wrong	How checked	Soup catches it?
Trained in another model's format, served in Qwen's: every row carries <code>< user >...</s></code> , which Qwen splits into plain characters, and TRL does not template string rows (<code>is_conversational = False</code>)	<code>audit_data.py</code> , TRL	No: <code>data validate "500/500 valid"</code> ; <code>data doctor</code> refuses DPO data
Memorised, not generalised: 81% train vs chance-level held-out	verify	No
Pre-flight admits a DPO run that does not fit (17-23% low)	measured vs panel	It is the check
fp16 overflow silently skips a step (step 11)	parsed every step	No, only <code>grad_norm: nan</code>
A skipped check reports "OK" (<code>near_duplicates</code> , <code>datasketch</code> not in <code>[train]</code>)	lint table	Says OK
0.75.1 withholds 10% of data by default and DPO never evaluates it (fixed on main, #1313)	code, <code>dpo.py:141-218</code>	No (pinned <code>val_split: 0</code>)
<code>pip install</code> gives 0.72.4 (2026-08-03, before the T4 fixes) on today's Colab	<code>pip --dry-run</code>	No
<code>soup ship</code> judges a T4 model in bf16, which the card only emulates, not the fp16 it trained in	ship log	No
<code>soup ship</code> rests on one strict task win: it tied 0.95 → 0.95 and all 8 suites +0.0000, so one flipped item = SHIP	ship verdict	No noise floor by default

4. Verdict: DON'T SHIP

The run completed, the memory held, the adapter is real. None of that answers the question: on unseen pairs the model is no better than a length heuristic, it learned to prefer longer answers, and it trained in a prompt format serving never sends. `soup ship` also said DON'T SHIP, but only because a ceiling task metric tied; that agreement is a coincidence.

I tested the obvious fix (data in Qwen's conversational format, 250/250 length-balanced, same held-out pairs, `data/prepare_fixed_data.py`): it still fails (0.48, 26% on chosen-shorter). The length pull is not only in the data; summing DPO's log-ratio over every token plausibly rewards length by itself (a hypothesis I have not isolated). **Before the next run is worth shipping:** (1) keep the format fix; (2) a length-controlled objective (length-normalised log-ratios) and more than 500 pairs, with the balanced held-out rule as the release gate; (3) a generation-level eval on real tickets with a pairwise judge and a noise floor before `soup ship`; (4) pin Python 3.12 + `soup-cli==0.75.1`, and do not trust the DPO pre-flight for fit.

What surprised me / what concerns me. Training was the uneventful part; the surprises were around it: the task's own install line gives an old Soup, the pre-flight documented as never under-predicting was 23% low, my own first pass rule credited a shortcut, and fixing the data did not fix the length preference. Still concerning: I scored fixed texts, not generations; the labels have no provenance; and I did not measure streamed-vs-resident DPO gradients on this card, so a subtly wrong backward pass would look exactly like this run. AI use: `AI_USAGE.md`.